
SIGAI-SES v1.0.0 — Reporte de Estado Final

Securitas Colombia S.A.



SIGAI-SES v1.0.0 — Reporte de Estado Final

Estado: LISTO PARA PRODUCCION (Release Candidate)

[!IMPORTANT] **Ultima actualizacion:** 14 de Julio de 2026

Auditoria tecnica: Realizada el 14 de Junio de 2026

El sistema ha sido **verificado, refactorizado y optimizado** para su despliegue en produccion. Se realizo una auditoria completa de todos los modulos del Backend y Frontend, con correcciones de seguridad y consistencia UX aplicadas.

1. Backend: Modulos y Logica de Negocio

[!NOTE] Backend construido con **FastAPI + SQLAlchemy (Async)**, integrando un motor de base de datos robusto y servicios especializados.

Modulos Implementados — Backend

Modulo	Archivo	Estado	Cobertura
Autenticacion	auth.py, security.py	COMPLETO	Login, refresh, logout, register, me
Usuarios	users.py, crud_user.py	COMPLETO	CRUD, auditoria, settings, avatar, cambio password
Inventario	inventory.py, crud_inventory.py	COMPLETO	Items, activos, ubicaciones, desmonte-bulk, EPP

Modulo	Archivo	Estado	Cobertura
Negocio	business.py, crud_business.py	COMPLETO	Clientes, proyectos, proveedores, garantias, movimientos, actas
Analiticas	analytics.py, crud_analytics.py	COMPLETO	Summary dashboard, search global
Reportes	reports.py	COMPLETO	Exportacion Excel/PDF con streaming (5 modulos)
Alertas	alerts.py, crud_alerts.py	COMPLETO	CRUD alertas, resumen, evaluacion
Regionales	regionales.py	COMPLETO	Listado de regionales
Importacion	import_data.py, im- port_service.py	COMPLETO	Excel con upsert (3 formatos)
Monitoreo	monitoring.py	COMPLETO	Health check, health/db, metrics

Motor de Importacion Inteligente

Caracteristica	Estado
Procesamiento de Excels masivos (Inventario, Garantias, Clientes)	Completado
Logica Upsert para evitar duplicados y normalizar datos tecnicos	Completado
Deteccion automatica de tipo por nombre de archivo	Completado
Procesamiento transaccional (todo o nada)	Completado

Sistema de Alertas y Notificaciones

Característica	Detalle
Motor de reglas implementado	stock_bajo (crítica) y garantia_vencida (alta)
Evaluacion automatica cada 30 minutos (+ al arrancar)	APScheduler (AsyncIOScheduler)
Estados	activa -> reconocida -> resuelta / ignorada
Gestion completa via CRUD endpoints	Completado

Auditoria y Trazabilidad

- Registro detallado de cada accion del usuario (CREATE, UPDATE, DELETE, LOGIN)
- Almacenamiento de valores “**Anterior**” y “**Nuevo**” para recuperacion de historial
- Auditoria **obligatoria** en todas las operaciones CRUD

Seguridad

Medida	Implementacion	Estado
Autenticacion	OAuth2 con JWT (Access 8h + Refresh 7d)	Completado
Encriptacion	Contraseñas con bcrypt	Completado
CORS	Configurable desde variable de entorno	Completado
Rate limiting	SlowAPI (10 req/min en login)	Completado
Soft delete	Tablas criticas (items, clientes, proveedores)	Completado

2. Frontend: Interfaz y Experiencia de Usuario

[!TIP] Desarrollado con **React 18**, **TypeScript 5.2** y **Tailwind CSS 3.4**, consumiendo la API mediante **Axios** con interceptores JWT.

Modulos Implementados — Frontend

Modulo	Componente	Estado	Acceso
Login	Login.tsx	COMPLETO	Publico
Dashboard	Dashboard.tsx	COMPLETO	Autenticado
Inventario	Inventory.tsx	COMPLETO	Autenticado
Cientes	Clients.tsx	COMPLETO	Autenticado
Proyectos	Projects.tsx	COMPLETO	Autenticado
Garantias	Guarantees.tsx	COMPLETO	Autenticado
Alertas	Alerts.tsx	COMPLETO	Autenticado
Desmontes	Desmontes.tsx	COMPLETO	Autenticado
Usuarios	Users.tsx	COMPLETO	Solo ADMIN
Auditoria	Audit.tsx	COMPLETO	Solo ADMIN
Entregas	Deliveries.tsx	COMPLETO	Solo ADMIN

Diseno — Fusion UI

[!NOTE] Sistema de diseno propio **“Fusion UI”** (Emerald Core x Neomorphic Hub)

Componente	Descripcion
Paleta	Oscura con acentos verde esmeralda
Tipografia	JetBrains Mono (monospace)
Componentes	Card, Button, Modal, ConfirmModal, Table, Badge, NeoInput, NeoSelect, DonutChart

Componente	Descripción
Sidebar	Colapsable y responsiva
Busqueda global	Con debounce de 400ms

Funcionalidades Clave

Funcionalidad	Estado
Paginacion real en todos los listados	Completado
Filtros por categoria, estado, busqueda textual	Completado
Importacion Excel con arrastrar y soltar	Completado
Notificaciones toast con animaciones CSS	Completado
Exportacion a PDF/Excel por modulo	Completado

3. Automatizacion y Mantenimiento

Control de Versiones

Practica	Implementacion	Estado
.gitignore optimizado	Desarrollo y produccion	Completado
.env.example	Backend y Frontend	Completado
Secrets saneados	Archivos .env sin credenciales	Completado

Dependencias

Componente	Detalle	Estado
requirements.txt	Versiones estables y categorizadas	Completado
package.json	Dependencias exactas	Completado
Dockerfile multi-etapa	Backend (python:3.12-slim) + Frontend (nginx:alpine)	Completado

Rendimiento

Medida	Valor	Estado
Paginacion	50 items/pagina	Completado
Streaming Excel	+30,000 registros	Completado
Pool de conexiones MySQL	30 conexiones, max_overflow=50, timeout 60s, recycle 1800s	Completado
GZip compression	Respuestas > 500 bytes	Completado

4. Hallazgos de Auditoria (14 Junio 2026) y Correcciones Aplicadas

H1: Secretos en repositorio — CORREGIDO

Campo	Detalle
Detectado	Archivos .env con SECRET_KEY, ADMIN_PASSWORD en texto plano en 3 ubicaciones (root, Backend, Frontend)

Campo	Detalle
Accion	Archivos saneados, creados <code>.env.example</code> como plantillas
Estado	CORREGIDO

H2: CORS permisivo — CORREGIDO

Campo	Detalle
Detectado	Backend/app/main.py tenia <code>allow_origins=['*']</code>
Accion	CORS ahora configurable desde variable <code>CORS_ALLOWED_ORIGINS</code> . Por defecto solo <code>http://localhost:5173</code>
Estado	CORREGIDO

H3: Credenciales en locustfile.py — CORREGIDO

Campo	Detalle
Detectado	<code>payload</code> con usuario y contraseña de prueba (<code>admin123</code>) en texto plano
Accion	Ahora usa variables de entorno con valores por defecto no sensibles
Estado	CORREGIDO

H4: UX inconsistente (window.confirm) — CORREGIDO

Campo	Detalle
Detectado	Confirmaciones dispersas con <code>window.confirm</code> en lugar del modal del sistema de diseno

Campo	Detalle
Accion	ConfirmModal centralizado en Fusion.tsx; reemplazos en Guarantees, Inventory, Projects, Users, Deliveries
Estado	CORREGIDO

H5: Logs y debug prints — CORREGIDO

Campo	Detalle
Detectado	console.log en Users.tsx, posibles prints en artefactos dist
Accion	Eliminados y corregida llamada fetchUsers() -> fetchData()
Estado	CORREGIDO

H6: Lazy loading en relaciones async — CORREGIDO

Campo	Detalle
Detectado	Posibles errores de sesion cerrada con lazy loading en SQLAlchemy async
Accion	Se usaron selectinload en crud_deliveries y getters criticos
Estado	CORREGIDO

H7: Tests y CI — CORREGIDO

Campo	Detalle
Detectado	Suite de tests parcial; habia pytest.ini pero faltaba CI automatizado

Campo	Detalle
Accion	Pipeline CI implementado en <code>.github/workflows/main.yml</code> (backend-tests con pytest sobre SQLite async + frontend-lint con ESLint) y pre-commit hooks
Estado	CORREGIDO

Resumen de Hallazgos

Hallazgo	Gravedad	Estado
H1 — Secretos en repositorio	Critico	Corregido
H2 — CORS permisivo	Alto	Corregido
H3 — Credenciales en locustfile	Alto	Corregido
H4 — UX inconsistente	Medio	Corregido
H5 — Logs y debug prints	Medio	Corregido
H6 — Lazy loading async	Medio	Corregido
H7 — Tests y CI	Bajo	Corregido

[!WARNING] **Los 7 hallazgos** fueron corregidos. El pipeline CI (tests + lint) y los pre-commit hooks están operativos.

5. Deuda Tecnica y Roadmap

v1.0.0 (Actual) — Release Candidate

Funcionalidad	Estado
Funcionalidad completa operativa	Completado

Funcionalidad	Estado
Importacion Excel con upsert y deteccion automatica	Completado
Reportes con streaming para +30k registros	Completado
Autenticacion JWT con refresh token	Completado
Auditoria completa con valores anterior/nuevo	Completado
Motor de alertas automatico	Completado
Desmontes con triaje	Completado
Sistema de regionales completo	Completado

v1.1.0 (Planificado — Proximo Sprint)

#	Tarea	Tipo
1	Notificaciones Push/Email — Integracion SMTP para notificar administradores	Nueva
2	Cobertura de Tests — Ampliar pruebas unitarias y de integracion	Calidad
3	Offline-ready (PWA) — consultas en campo sin conexion	Mejora
4	Security Headers — CSP, HSTS, X-Frame-Options	Seguridad
5	Bloqueo por intentos fallidos de login	Seguridad

#	Tarea	Tipo
6	Rate limiting general (no solo en login)	Seguridad

v1.2.0 (Planificado)

#	Tarea	Tipo
1	Autenticacion 2FA	Seguridad
2	Monitoreo con Sentry	DevOps
3	Encriptacion de credenciales tecnicas de activos (AES-256)	Seguridad
4	Pruebas de penetracion externas	Seguridad

6. Conclusion

[!NOTE] La aplicacion **SIGAI-SES v1.0.0** cumple con el **100%** de los requisitos funcionales de gestion de inventario, auditoria y alertas. La auditoria de seguridad encontro **7 hallazgos**, los **7 fueron corregidos** (incluido el pipeline CI). Es una base **solida y escalable** para la operacion de Seguridad Electronica Securitas.

Cumplimiento de Requisitos

Indicador	Valor	Progreso
Requisitos Funcionales	21 de 21 (100%)	
Requisitos No Funcionales	5 de 5 (100%)	
Historias de Usuario	24 de 24 (100%)	

Etiquetas del Proyecto

Tag	Valor
Version	v1.0.0-RC
Ultima actualizacion	14 Julio 2026
Auditoria	14 Junio 2026
Backend	FastAPI · SQLAlchemy Async · Python 3.12
Frontend	React 18 · TypeScript 5.2 · Tailwind CSS 3.4
Diseno	Fusion UI (Emerald Core x Neomorphic Hub)
Deploy	Render · Supabase (PostgreSQL) · Docker · On-premise
Auditorias corregidas	7 / 7

Acciones Inmediatas Recomendadas

[!WARNING] **Antes del despliegue en produccion**, asegurese de completar las siguientes acciones:

1. **Rotar secrets** y asegurar `.env` en servidores de produccion
2. **Ejecutar carga inicial** con `import_t_data.py` con los Excels oficiales finales
3. **Configurar despliegue** con Render + Supabase (PostgreSQL) u on-premise (ver GUIA_DESPLIEGUE_PRODUC)
4. **Configurar Nginx** como reverse proxy con HTTPS (Let's Encrypt)

Documento actualizado al: **Julio 2026**